

Why the temporal split scores zero

The prediction target is constant within a reporting quarter. This report establishes what the models learn instead, what the reported metrics measure, and which parts of the work stand independently of the problem.

REPOSITORY

chaiv/tweetsCompanyNumbersPrediction

ENTRY POINT EXAMINED

trainNumbersPredictionModelTemporalSplit.py

DATASETS

Apple EPS · Amazon revenue · Tesla car sales

PERIOD

2015Q1 – 2019Q4

DATE

17 August 2026

The three results that carry the report

- 1. Each dataset contains only **20 distinct labels** — one per reporting quarter — for more than a million tweets. Tweet-group text identifies its own quarter with **91.8 % accuracy** across those 20 classes.
- 2. A seasonal rule that reads *no tweets at all* reaches **accuracy 0.846 and MCC 0.769** on Apple EPS, and **1.000 / 0.999** on Amazon revenue. The thesis abstract reports 0.87 and 0.77.
- 3. Out of period the trained model reproduces the *previous* quarter's label in **12 of 12 quarters**, on two independent datasets.

Separately, the LSTM in its current form does not fit its training data in any configuration tested, and every training script silently scores the previous run's checkpoint. Both were fixed and verified; neither changes the conclusions above.

1 Summary

The strict temporal split is not a weaker variant of the other evaluation protocols. It is the only protocol in the repository that closes a shortcut the others leave open, and very little survives once that shortcut is closed.

Every tweet is labelled with the change of the financial figure of the reporting period it falls into. The label is therefore constant inside a quarter, and five years of data contain twenty distinct labels rather than a million. The consequence is that the mapping from text to label factorises entirely through the quarter: identify the period a text came from and the label follows from a lookup table with twenty entries.

Tweet text identifies its period very well, because product names, prices, news events and hashtags are period markers. Any split that places groups from the same quarter into training and test is therefore solvable without any predictive content, and the repository's main evaluation does exactly that for 100 % of its test groups. Under protocols that keep periods disjoint, the measured correlation with the truth is approximately zero on all four datasets.

Two defects unrelated to the split were found while verifying this and are documented in section 4: the LSTM in its current form does not fit its training data in any configuration tested, including that of the main evaluation, and every training script rebuilds its checkpoint filename by hand, so a second run silently scores the model produced by the first. Repairing the first defect makes the picture sharper rather than softer. The corrected model reaches accuracy 0.907 and MCC 0.815 on data that shares quarters with training — the range the abstract reports — and an MCC between 0.15 and 0.29 on quarters it has not seen.

None of this means tweets carry no information about company finances. It means the present experimental design cannot answer that question, because it offers at most twenty independent cases to be right or wrong about, and because knowing the date is nearly equivalent to knowing the answer. Section 5 sets out the parts of the work that stand on their own, and section 6 the changes that would let the question be asked properly.

2 What the models do in their current form

2.1 The pipeline as built

Tweets for one NASDAQ ticker are joined to a table of quarterly figures by `TweetNumbersConnector`, which selects the row satisfying `from_tsp ≤ post_tsp ≤ to_tsp` — the reporting period containing the tweet. The `percent_change` of that row is discretised into two or four classes. Tweets of the same class are then chunked into groups of N (5, 10 or 20) by `DataframeSplitter.getSplitIds`, and each group becomes one training sample whose text is the concatenation of its tweets separated by a `<SEP>` token.

The classifier is LSTMNN: a 300-dimensional embedding initialised from Top2Vec word vectors and left trainable, a two-layer LSTM with hidden size 512, then linear layers $512 \rightarrow 512 \rightarrow 256 \rightarrow$ number of classes with ReLU between them. Training runs ten epochs of Adam at learning rate $1e-3$ under PyTorch Lightning with mixed precision, inverse-frequency class weights, early stopping on validation loss, and checkpointing on the best validation loss.

2.2 What the trained model actually computes

Empirically, two things, in this order of importance.

It recognises the period a text belongs to. A bag-of-words model over the same tweet groups assigns them to the correct one of twenty quarters with 91.8 % accuracy, against a chance level of 5 %. Because a quarter determines the label, this recognition is sufficient to produce a high score whenever training and test share quarters.

Out of period, it returns the label of the nearest period it has seen. Tweet vocabulary drifts smoothly over time, so the training quarter that most resembles an unseen quarter is the one immediately preceding it. Trained walk-forward, the model's majority prediction per test quarter equalled the previous quarter's label in twelve of twelve quarters on Apple EPS and, independently, twelve of twelve on Amazon revenue. It does not anticipate a change; it echoes the most recent one.

The network does not fit its training data

Run unchanged on Apple EPS, the training loss remains at 1.3864 from the first epoch to the tenth. Since $\ln(4) = 1.3863$ and the loss is class-weighted by inverse frequency, this is exactly the loss of a model that emits the same distribution for every input — which is what it does, predicting class 0 for all 28,501 test groups. Five optimiser configurations were trained on identical cached data (mixed precision on and off, learning rates $1e-3$, $3e-4$ and $1e-4$, gradient clipping, frozen embeddings) and all collapsed identically. The cause is that `forward` reduces a sequence of up to 320 stemmed tokens to `h_n[-1]`, its final hidden state.

This is not confined to the temporal split's data. The same failure occurs in the configuration of the *main* evaluation — `BALANCE_CLASSES=True`, ten-fold rotating temporal blocks per class — on both datasets tested, as section 2.3 shows.

2.3 The main evaluation's configuration, with and without the pooling fix

The data path of `trainNumbersPredictionModelStratifiedKFoldTemporalPerClass.py` was reproduced exactly (`EqualClassSampler`, temporally sorted groups, fold 0 of the rotating per-class blocks) and the unmodified architecture trained on it, then the same architecture with mean pooling.

CONFIGURATION	FINAL TRAIN LOSS	VAL ACCURACY	VAL MCC	TEST MCC
Apple EPS, 4 classes — repository model	1.331	0.238	-0.024	0.024
Apple EPS, 4 classes — mean pooling	0.008	0.866	0.821	0.625
Amazon revenue, binary — repository model	0.690	0.519	0.131	0.285
Amazon revenue, binary — mean pooling	0.005	0.907	0.815	0.218

Fold 0 of the main evaluation's split, groups of 10. Validation is a random sample of the training blocks and therefore shares quarters with training, which is the situation the reported metrics were measured in; the test column is fold 0's held-out block. $\ln(2) = 0.693$ and $\ln(4) = 1.386$.

Two things follow. First, the repository model does not fit its training data in the main evaluation's configuration either: its loss sits at the entropy of the class prior on both datasets, and on Apple its test accuracy of 0.253 against four balanced classes is exactly chance.

Second, and more useful, the corrected model reproduces the reported performance range precisely where the theory predicts. On data that shares quarters with training it reaches accuracy 0.907 and MCC 0.815 on Amazon, and 0.866 and 0.821 on Apple — the abstract reports 0.87 and 0.77. On quarters it has never seen, the same trained model falls to an MCC of 0.15 to 0.29. This is the report's argument in a single table: the headline numbers are what a working model produces when the periods overlap, and they do not survive when the periods are separated.

2.4 What each evaluation protocol measures

The decisive quantity is the share of test groups whose quarter also appears in training. Where that share is high, the protocol measures period recognition. Where it is zero, it measures transfer to unseen periods.

PROTOCOL	TEST QUARTERS SEEN IN TRAINING	ACCURACY	MCC	MEASURES
10-fold, rotating temporal block per class (main evaluation)	100 %	0.659	0.499	period recognition
Strict 80/20 temporal split	0 %	0.367	0.196	transfer
Leave one quarter out, all 20 quarters	0 %	0.395	0.016	transfer
Walk forward, train on all earlier quarters	0 %	0.377	-0.035	transfer

Apple EPS, groups of 10, measured with a TF-IDF and logistic-regression stand-in so that every protocol could be run repeatedly. The stand-in reproduces the pattern of the reported results; its absolute values sit below those of the LSTM with pretrained embeddings.

Walk-forward MCC is likewise at or below zero on the other datasets: -0.172 on Amazon revenue binary, -0.123 on Amazon revenue four-class, -0.009 on Tesla car sales. Across four datasets, no out-of-period

protocol produces a positive correlation with the truth.

3 The structure of the target

Each cell below is the class assigned to every tweet of that quarter. Reading down the columns shows the label to be a function of the calendar quarter almost everywhere. Amazon revenue under the binary scheme is perfectly periodic across all five years.

Apple EPS, 4 classes

	Q1	Q2	Q3	Q4
2015	0	0	1	3
2016	0	0	2	3
2017	0	0	2	3
2018	0	0	2	3
2019	0	0	3	3

Amazon revenue, binary

	Q1	Q2	Q3	Q4
2015	1	0	1	1
2016	1	0	1	1
2017	1	0	1	1
2018	1	0	1	1
2019	1	0	1	1

Amazon revenue, 4 classes

	Q1	Q2	Q3	Q4
2015	3	0	1	1
2016	3	0	1	1
2017	3	0	1	2
2018	3	0	1	1
2019	2	0	1	1

Tesla car sales, 4 classes

	Q1	Q2	Q3	Q4
2015	1	1	1	3
2016	0	0	3	0
2017	1	0	2	1
2018	1	3	3	3
2019	0	3	1	2

0 decrease 1 weak increase 2 moderate increase 3 strong increase binary scheme: 0 decrease, 1 increase

3.1 The calendar alone reproduces the reported result

A rule with no text input — predict the label of the same calendar quarter observed in earlier years — was evaluated walk-forward over every quarter that has at least one earlier year available.

DATASET	ACCURACY	MCC	QUARTERS CORRECT
Amazon revenue, binary	1.000	0.999	16 / 16
Amazon revenue, 4 classes	0.872	0.826	14 / 16
Apple EPS, 4 classes	0.846	0.769	14 / 16
Tesla car sales, 4 classes	0.225	-0.077	4 / 16

Seasonal naive forecaster, walk forward over 16 quarters per dataset. Produced by `evaluateTemporalBaselines.py`, which trains nothing and requires no GPU.

The thesis abstract reports an accuracy of 0.87 and an MCC of 0.77. On Apple EPS the seasonal rule reaches 0.846 and 0.769 without reading a single tweet. Tesla is the informative exception: its car sales grew close to monotonically over the period, so the quarter-on-quarter changes are not seasonal, the rule fails, and the text models fail there too.

Text cannot substitute for the calendar either. A classifier trained to recognise the *calendar* quarter from tweet text, with year-specific vocabulary removed by requiring tokens to occur in at least four different years with no more than 35–45 % of their occurrences in any single year, transfers to a later year at 0.35–0.43 accuracy against a chance level of 0.25. That is far too weak to drive the seasonal mapping from text.

4 Defects found while verifying

These are independent of the evaluation design and would need fixing whatever target the system predicts.

D1 The LSTM does not fit its training data

Documented in sections 2.2 and 2.3, on Apple EPS and Amazon revenue, in both the temporal configuration and the main evaluation's balanced one. The fix is to pool over the sequence rather than take the last hidden state. Replacing `h_n[-1]` with a mean over the LSTM outputs at non-padding positions, with every layer, the learning rate and the data left untouched, drives the training loss from 1.386 to 0.008 and the model begins separating all classes.

PROTOCOL	ACCURACY	MCC
Random split — test quarters shared with training	0.800	0.687
Strict temporal split — test quarters unseen	0.571	0.286
Strict temporal split, full Lightning pipeline end to end	0.463	0.148

Identical architecture, data and training; the only difference is whether the test quarters were also present in training.

The fix therefore lifts the temporal split from a degenerate 0.000 to a genuine 0.148, which remains far below the 0.764 the calendar achieves on the same test period. Repairing the model does not recover the reported performance, because the reported performance never came from the model's reading of the tweets.

D2 Re-running a training script scores the previous run's model

Lightning's `ModelCheckpoint` appends a version suffix when the target file already exists, so a second run writes `tweetpredict_fold0-v1.ckpt` and leaves `tweetpredict_fold0.ckpt` in place. Every training script then rebuilds the path by hand from `checkpointName` and loads the older file. Because `loadModel` calls `load_state_dict(..., strict=False)`, the mismatch raises nothing.

This was observed directly: the first run of the corrected script reported accuracy 0.44 and MCC 0.0 — the collapsed model of the run before it — while Lightning's own test loss on the newly trained model was 1.088, already below $\ln(4)$. The printed number and the trained model were different objects, and nothing in the output indicated it. Any result obtained by re-running a script into a directory that already held checkpoints is worth re-checking.

D3 The label describes the tweet's own quarter

The joined `percent_change` is the change of the period containing the tweet against the preceding one, so a tweet from March 2015 carries “Q1 2015 EPS fell 24.7 % against Q4 2014”. Since the figure is published only after the quarter closes, this is a defensible nowcast of an unannounced number, but it is not the prediction of a subsequent metric that the README describes, and it is the reason that identifying the quarter is equivalent to knowing the answer.

D4 Epoch seconds parsed as nanoseconds

`post_date` holds epoch seconds and the training scripts call `pd.to_datetime` without `unit="s"`, so pandas reads the integers as nanoseconds and maps all 1.4 million tweets to 1 January 1970. The relative order survives, which is why the temporal sorting still works and the defect has been invisible, but every calendar-based operation on that column is wrong.

5 Where the current work is useful or interesting

The diagnosis above concerns one claim: that the models predict financial change from the semantics of social media text. Several other contributions in this work are untouched by it, and one of them is arguably stronger than the original claim.

5.1 Financial Twitter carries a sharp temporal fingerprint

The strongest empirical result in the repository is one that was not being claimed. Aggregated tweet groups about a single company identify their own quarter with 91.8 % accuracy across twenty classes, from stemmed unigrams alone. Ten tweets are enough to place a conversation within a three-month window over a five-year span. That is a substantive and measurable property of financial social media, it is reproducible from the existing pipeline without modification, and it is the mechanism behind every high score the system has produced. Presented directly — as temporal identifiability rather than financial prediction — it is a defensible result with a clean experimental design already in place.

5.2 The seasonality of the metrics is a real finding in its own right

That Amazon's quarterly revenue falls in Q2 and rises in every other quarter, in each of five consecutive years without exception, is an economically meaningful regularity, and the same holds for the Apple EPS cycle of two declining quarters followed by two rising ones. The seasonal baseline built for this diagnosis quantifies it: MCC 0.999 for Amazon binary, 0.826 for Amazon four-class, 0.769 for Apple. This is exactly the kind of result a forecasting study should report, and its absence is what allowed the text model's score to be read as evidence of something else.

5.3 The interpretation pipeline works — on a different question

The Integrated Gradients attribution, the Top2Vec and BERTopic mapping and the important-word extraction are mechanically sound and unaffected by the defects above. What they explain, however, is the basis on which the model discriminates, and that basis is the period. Read as “which terms and topics most sharply characterise a given quarter of discussion about this company”, the outputs become interpretable and correct. The Black Lives Matter and Christmas campaign association, and the Brexit and cryptocurrency shift, are convincing as descriptions of what dominated a period's conversation. They are not evidence of a causal channel into the financial figure, but they are genuine period-characteristic topic extraction, which is a legitimate contribution to corpus analysis.

5.4 The system is a reusable instrument

The engineering contribution stands independently of the target: joining a tweet corpus to a table of reporting periods, deriving classes from percentage change, forming aggregate groups, exporting topic-model vocabulary as pretrained embeddings, training and checkpointing, attributing predictions to tokens, mapping tokens back to topics, and comparing against an LLM. Swapping the target — for instance to a share-price return measured over the days following each tweet group — requires changes to one stage,

not to the architecture. The thesis describes a working system for predicting organisational metrics from social data; that system exists and runs, and the diagnosis concerns what it was pointed at.

5.5 The negative result is publishable, and possibly more valuable

Labelling every document of a reporting period with that period's outcome is a common construction in the social-media-to-financials literature, and the shortcut it creates is rarely measured. This work now contains an unusually clean demonstration of it on real data: the quantified period-recognition rate, the 100 % quarter overlap in a superficially careful stratified cross-validation, the collapse to lag-1 persistence in twelve of twelve quarters on two independent datasets, and a no-text baseline that matches the headline metrics. A methodological paper built on that — how period identification masquerades as prediction, and which diagnostics expose it — would be a contribution to a literature that needs it, and it rests on experiments that have already been run.

5.6 The nowcasting question remains open and askable

Because a quarterly figure is announced only after the quarter closes, a model reading tweets from inside quarter Q to infer Q 's unannounced figure is performing a legitimate nowcast, and this is a recognised task in the finance literature. The present results neither establish nor refute it; the design forecloses the test. With a walk-forward protocol, a seasonal baseline reported alongside, and a target sampled more finely than once per quarter, the same data and the same pipeline could ask the question properly.

6 Recommendations

6.1 Reporting

- **Report the seasonal baseline beside every model score.** On the three seasonal datasets it is the number to beat and it is currently not beaten. This single addition changes what the reported metrics can be claimed to show, and it is one script run.
- **State the quarter overlap of every split.** A protocol whose test quarters also appear in training measures period recognition; the reader cannot tell from the name of the split alone.
- **Report the number of independent cases, not only the number of tweet groups.** A test block of six quarters decides six things, however many groups it contains, and confidence intervals should be stated on that basis.

6.2 Experimental design

- **Give the target more than twenty values.** This is the change that matters most. A target sampled at tweet frequency — for example the return of the share price over the days following a tweet group — turns twenty decisions into thousands and makes an out-of-period protocol statistically meaningful.
- **Shift the target forward.** Labelling the tweets of period P with the change of period P+1 makes the task genuinely predictive and breaks the equivalence between recognising the period and knowing the answer.
- **Evaluate walk-forward rather than with a single cut.** One 80/20 split decides five or six quarters; a rolling origin over all available quarters gives a far more stable estimate at similar cost.

6.3 Code

The following changes were made and verified during this diagnosis.

FILE	CHANGE
<code>classifier/TemporalSplitDiagnostics.py</code>	New. Quarter composition of a split, train-test quarter overlap, and the majority, persistence and seasonal baselines. Parses <code>post_date</code> as epoch seconds.
<code>evaluateTemporalBaselines.py</code>	New. Runs those baselines over all four experiments and adds a walk-forward seasonal evaluation. No training, no GPU.
<code>classifier/LSTMNN.py</code>	New pooling option. 'last' remains the default; 'mean' averages the LSTM outputs over non-padding positions.
<code>classifier/CreateClassifierModel.py</code> <code>classifier/ModelEvaluationHelper.py</code>	Pass pooling and the pad token id through, so a checkpoint is always evaluated with the pooling it was trained with.
<code>classifier/Trainer.py</code>	<code>train</code> returns <code>checkpoint_callback.best_model_path</code> .
<code>trainNumbersPredictionModelTemporalSplit.py</code>	Mean pooling; the checkpoint path from the trainer; validation taken from the latest part of the training period rather than a random stratified sample; prints split composition, baselines and a per-quarter comparison against the previous quarter's label.

Working tree of the repository; the model default is deliberately left unchanged so that no existing experiment or checkpoint is altered silently.

The other four training scripts still rebuild their checkpoint filenames by hand and still parse `post_date` without a unit. Both fixes apply unchanged.

7 Methods and scope

7.1 How the numbers were produced

Tweet groups were rebuilt exactly as `DataframeSplitter.getSplitIds` forms them — consecutive chunks of N tweets of the same class — then sorted by the earliest tweet in each group, matching the training scripts. Protocol comparisons used a TF-IDF representation with a logistic-regression classifier so that leave-one-quarter-out and walk-forward could each be run sixteen to twenty times; the LSTM results were produced with the repository architecture on the same cached token sequences, and end to end through the unmodified Lightning pipeline where stated. Vocabulary was always fitted on training data only. All figures are on the full datasets in `companyTweets`, groups of ten tweets, 2015Q1 to 2019Q4.

7.2 What this diagnosis does not establish

- It does not show that tweets contain no information about company finances. It shows that this design cannot demonstrate such information, and that the reported metrics do not.
- The protocol comparison in section 2.3 used a linear stand-in, not the LSTM. Its role is the *ordering* of the protocols, which is large and consistent; the absolute values would differ for the LSTM.
- The 0.148 achieved on the strict temporal split after the pooling fix rests on six test quarters and should be read as evidence that the model now functions, not as a precise estimate of its skill.
- Defect D1 was established on Apple EPS and on Amazon revenue binary, in both the temporal configuration and the main evaluation's balanced configuration. Tesla and the remaining experiments were not tested, and neither were the group sizes 5 and 20.
- Section 2.3 reproduces fold 0 of the main evaluation, not all ten folds, and its training used plain fp32 Adam rather than the Lightning loop with early stopping. It shows that the reported performance range is reachable on quarter-sharing data and not on disjoint data; it is not a re-run of the thesis experiment.
- Whether the published figures were themselves affected by defects D1 and D2 cannot be determined from the repository alone, since neither leaves a trace in the recorded output.

7.3 Reproducing

From `tweetsCompanyNumbersPrediction/src,python evaluateTemporalBaselines.py` reproduces the quarter-label tables of section 3, the quarter overlap of each split and the baselines of section 3.1 in a few minutes on CPU. `python trainNumbersPredictionModelTemporalSplit.py` reproduces the corrected temporal result and prints the diagnostics and the per-quarter comparison.